PROJECT PROGRESS DOCUMENTATION

Applicant Management System

GROUP 7

PROJECT TEAM

MEMBER	
Flordeliza, Tomas Josh	
Sarmiento, Denzel Gabriel	
Ibias, Carl Ivan	
Romero, John david	
Calcita, Austin John	

Repository: https://github.com/mang2mas/grp07-applicant-management-system.git

Progress tracker: [Insert tracker / Google Sheets URL]

Course / section: BSIT 3-2 Submission date: 11/06/2026

1. System Overview

Purpose. The Group 7 Applicant Management System (AMS) is a PHP web application intended to centralize applicant and recruitment records. It helps staff maintain applicant profiles, organize departments and positions, record applications, monitor requirements, and manage interview schedules and outcomes.

Scope. The documented workflow covers organizational setup and applicant registration through application processing, requirement tracking, and interview management. The project modules discussed during development include Dashboard, Applicants, Departments, Positions, Applications, Requirements, Applicant Requirements, Interviews, and System Status.

Main functions:

- Create, view, update, search, and delete applicant records.
- Maintain departments and their job positions.
- Create applications linking applicants to positions and application statuses.
- Maintain required/optional document catalogues and assign requirements to applicants.
- Track requirement status, submission date, and remarks.
- Schedule interviews and record type, result, and notes.
- Review records through dashboard summaries, searchable lists, and pagination.

Alignment with approved Activity 2 proposal: [Insert the exact approved purpose, scope, objectives, and functions. Confirm this overview matches the proposal.]

1.1 Module Summary

MODULE	FUNCTION
Dashboard	Overview of key records and recruitment activity.
Applicants	Applicant profile and contact information management.
Departments	Department records used to group positions.
Positions	Job title, department, description, employment type, and status.
Applications	Applicant-to-position records with application date and status.
Requirements	Master list of required and optional applicant documents.
Applicant Requirements	Per-applicant assignment and tracking of status, date, and remarks.
Interviews	Interview schedules, types, results, and notes.
System Status	System/database status information.

1.2 Technology and Data Storage

The system uses PHP for the web application and PostgreSQL for relational data storage. Database setup is separated into db/01_schema.sql (schema, relationships, and constraints) and db/02_seed.sql (fictional sample data for testing).

2. User Roles and Data Access

The roles below describe intended operational responsibilities. Verify the final source code before claiming that authentication or permission restrictions are implemented.

ROLE	RESPONSIBILITIES
Administrator	System owner or database administrator responsible for maintaining the system, managing database configuration, and performing full create, read, update, and delete (CRUD) operations across all application screens.
Staff	HR staff or recruiters responsible for day-to-day recruitment activities, including managing departments and positions, registering applicants, processing applications, tracking requirements, and scheduling interviews. Has full CRUD access across application screens.
Viewer	Auditors and reviewers who can browse and view records across all application pages. Has read-only access and cannot create, edit, or delete records.
Application runtime (system account, not a person)	The database account used by the PHP backend through the DB_USER setting in .env. Its database access is associated with the permissions assigned to the application runtime role.

2.1 Database Roles and Accounts

The PHP application connects to PostgreSQL through project database configuration/environment settings. Fill in the actual database role and privileges. Do not include its password.

DATABASE ACCOUNT / ROLE	PURPOSE / PRIVILEGES
`ams_admin` — Member of `app_admin_grp`	Used for database administration and assigned the privileges of the administrator group role.
`ams_app_user` — Member of `app_runtime_grp`	Used by the PHP backend through the `DB_USER` setting in `.env`. Provides database access required by the application runtime.
`ams_viewer` — Member of `app_readonly_grp`	Used for read-only reporting and auditing. Intended to allow viewing database records without modifying them.

2.2 Role-to-Data Mapping

Recruitment operations use applicant, department, position, application, application_status, requirement, applicant_requirement, and interview records. Confirm whether these responsibilities are enforced by application authentication/authorization or are operational responsibilities only.

3. Development Progress Log

Enter dated milestones or weekly entries. Identify what was built/tested, who worked on it, the outcome or issues, and the related Git commit or hash.

Suggested format: date/week | milestone/task | contributor(s) | outcome/issues | commit hash or repository link.

DATE / WEEK	MILESTONE / WORK COMPLETED	CONTRIBUTOR / COMMIT / NOTES

4. Database Implementation

4.1 Schema and SQL Files

- db/01_schema.sql: defines relational tables, primary/foreign keys, uniqueness rules, check constraints, and indexes.
- db/02_seed.sql: provides fictional sample data for departments, positions, applicants, statuses, applications, interviews, requirements, and applicant requirements.

4.2 Main Tables and Relationships

TABLE	PURPOSE / RELATIONSHIP
department	Department records.
position	Job positions; references department.
applicant	Applicant profile information.
application_status	Available application status values.
application	Links applicant and position to a status.
interview	Interview schedule and outcome for an application.
requirement	Master list of applicant requirements.
applicant_requirement	Links applicants and requirements; stores submission tracking.

- Department → Position: one department can have multiple positions.
- Applicant + Position → Application: application connects an applicant to a position and status.
- Application → Interview: an application can have interview records.
- Applicant + Requirement → Applicant Requirement: tracks each assigned requirement.

4.3 Seed Data and Design Differences

- The inspected position status constraint permits only Open and Closed, while the Operations Supervisor seed row used On Hold. Change the sample value to a permitted value unless the approved design is changed.
- The inspected interview table uses column results, while the seed script used result. The final seed SQL and PHP queries must consistently use the actual column name.
- Seed execution failed because position.department_id = 2 was missing when positions were inserted. Reconcile existing department records/IDs and seed conflicts before rerunning.

4.4 ERD


public
department
department_id serial
department_name character varying(100)
description text
public
applicant
applicant_id serial
first_name character varying (50)
last_name character varying (50)
email character varying(100)
phone character varying(20)
address text
date_of_birth date
created_at timestamp without time zone
public
requirement
requirement_id serial
requirement_name character varying(100)
description text
is_required boolean
public
position
position_id serial
department_id integer
position_title character varying(100)
description text
employment_type character varying(30)
status character varying(20)
public
application_status
status_id serial
status_name character varying(50)
description text
public
applicant_requirement
applicant_id integer
requirement_id integer
submitted_date date
status character varying(20)
remarks text
public
application
application_id serial
applicant_id integer
position_id integer
status_id integer
application_date date
public
interview
interview_id serial
application_id integer
interview_date timestamp without time zone
interview_type character varying(30)
results character varying(30)
notes text

5. Application Evidence

Insert screenshots of the actual working system and CRUD test results. Use fictional test records where possible. Check screenshots for exposed passwords, private applicant details, or other secrets.

MODULE / EVIDENCE	WHAT TO SHOW
Dashboard	Dashboard overview and visible summaries.
Applicants	List and create/view/update/delete evidence.
Departments and Positions	Lists/forms and successful CRUD tests.
Applications	Application list/form and status updates.
Requirements	Requirement catalogue and required/optional values.
Applicant Requirements	Assignment, status/date/remarks update, and removal.
Interviews	List, create/edit, result update, and delete.
CRUD Test Results	Evidence for create, retrieve, update, and delete operations.

Paste screenshots under the relevant module headings or add them as an appendix.

6. Setup and Configuration

6.1 Prerequisites

- PHP installed and available in the terminal.
- PostgreSQL installed and running.
- An AMS database created in PostgreSQL.
- Project source code cloned or copied locally.
- Required PHP extensions, including the PostgreSQL PDO driver, enabled as required by the project.

6.2 Installation and Execution

1. Clone the group's repository or obtain the source code.
2. Open the project folder in Visual Studio Code.
3. Configure the environment file using .env.example, if provided.
4. Create the PostgreSQL database.
5. Execute db/01_schema.sql against that database.
6. Execute the corrected db/02_seed.sql if sample data is required.
7. Confirm database name, host, port, username, and password configuration matches the database used by PHP.
8. From the project directory, start the server (example): php -S localhost:8000 -t public.
9. Open http://localhost:8000 in a browser.

6.3 Environment Variable Placeholders

Replace these names with the exact variables in the team's .env.example if they differ. Do not include real values.

VARIABLE	SAFE DOCUMENTATION VALUE
DB_HOST	127.0.0.1
DB_PORT	5432
DB_NAME	grp07_applicantdb
DB_USER	your_db_user
DB_PASSWORD	your_password_here

6.4 Database User and Role Setup

USER ROLES AND DATABASE ACCESS CONTROL DOCUMENTATION

Role Mapping

The system implements three application-level user roles: Administrator, Staff, and Viewer. Each role has designated responsibilities and access permissions within the web application. These roles are mapped to corresponding PostgreSQL group roles to establish database-level access control.

Application Role	Intended User	Web Application Access	PostgreSQL Group Role
Administrator	System owner / database administrator	Full access to all application features	`app_admin_grp`
Staff	HR staff / recruiters	Full access to day-to-day operations	`app_runtime_grp`
Viewer	Auditors and reviewers	Read-only access to all application pages	`app_readonly_grp`

Application Runtime	PHP backend system account	Executes database operations according to application permissions	`app_runtime_grp`

Role Configuration Notes:

- Administrator and Staff currently have identical application permissions. They remain separate roles to support future Administrator-only features, such as user management.
- The `app_admin_grp` PostgreSQL role provides elevated database privileges but is not configured as a PostgreSQL superuser.
- The application runtime account shares `app_runtime_grp` with Staff, ensuring that the backend operates within the intended Staff-level database privileges.

Application Role Permissions

The following table describes the access permissions assigned to each application role.

Legend:

- R – Read / View
- CUD – Create, Update, and Delete
- – – No Access

Application Screen	Administrator	Staff	Viewer
Dashboard	R	R	R
Applicants	R + CUD	R + CUD	R
Applicant Requirements	R + CUD	R + CUD	R
Applications (including status changes)	R + CUD	R + CUD	R
Positions	R + CUD	R + CUD	R
Departments	R + CUD	R + CUD	R
Requirements	R + CUD	R + CUD	R
Interviews	R + CUD	R + CUD	R
System Status	R	R	R

Administrator and Staff can perform all available operations, while Viewer accounts are restricted to viewing information without modifying system records.

Application-Level Role Enforcement

The application implements role-based access control through authentication, session management, and server-side permission checks.

User Authentication

The `login_process.php` file handles user authentication. It retrieves the submitted username from the `users` table and verifies the supplied password against its stored BCRYPT hash using PHP's `password_verify()` function.

Upon successful authentication, the system stores the following information in the PHP session:

- User ID
- Username
- Assigned application role

The session ID is also regenerated after successful login to improve session security.

Session Protection

The `public/index.php` entry point checks whether a valid user session exists before allowing access to protected application pages and actions.

Only the login page and login processing action are accessible without an authenticated session. Unauthorized requests are redirected to the login page.

Viewer Restrictions

Viewer permissions are enforced through both server-side validation and interface restrictions.

Server-side enforcement:

- Viewer accounts receive an HTTP 403 Forbidden response when attempting POST actions, except logout.
- Viewer accounts cannot access create or edit form pages.
- Restrictions remain enforced even if a user manually submits a request instead of using the interface.

Interface-level enforcement:

- Create, Edit, Delete, Assign, and status-update controls are hidden from Viewer accounts.
- Viewer accounts can browse application pages and view available records.

Additional Security Measures

The application also implements the following security measures:

- CSRF tokens are included in application forms to help prevent Cross-Site Request Forgery.
- PDO prepared statements are used for SQL queries to help prevent SQL injection.

PostgreSQL Database Privileges

PostgreSQL group roles are designed to control database access according to the responsibilities of each account.

Privilege Legend:

- C – INSERT
- R – SELECT
- U – UPDATE

- D – DELETE
- – – No Access

Database Table	app_admin_grp	app_runtime_grp	app_readonly_grp
department	CRUD	CRUD	R
position	CRUD	CRUD	R
applicant	CRUD	CRUD	R
application_status	CRUD	R	R
application	CRUD	CRUD	R
interview	CRUD	CRUD	R
requirement	CRUD	CRUD	R
applicant_requirement	CRUD	CRUD	R
users	CRUD	R, U on password_hash only	–

Special Database Permissions

The application runtime account requires specific access to the `users` table to support authentication.

- SELECT permission allows the application to retrieve user records during login.
- UPDATE permission on the `password_hash` column allows the application to upgrade existing password hashes when the password hashing algorithm or cost configuration changes.
- The runtime account does not receive unrestricted UPDATE privileges on the entire `users` table.

Database Access Separation

The application uses a single database connection for authenticated users. Therefore, Viewer restrictions within the web application are enforced through PHP server-side authorization rather than separate PostgreSQL connections for each application user.

The `app_readonly_grp` role is intended for a separate database login, `ams_viewer`, which provides read-only access for direct database reporting and auditing.

PostgreSQL Database Login Accounts

The following database login accounts are designated for the system's database access configuration.

Database Login	Assigned Group Role	Purpose
`ams_admin`	`app_admin_grp`	Database administration
`ams_app_user`	`app_runtime_grp`	Database connection used by the PHP backend through the `.env` configuration
`ams_viewer`	`app_readonly_grp`	Read-only database reporting and auditing

These accounts separate administrative, application runtime, and reporting responsibilities.

Application Test Accounts

The following test accounts are defined in the `users` table through the `db/02_seed.sql` seed file.

Username	Application Role	Test Password
`admin`	Administrator	`password123`
`staff`	Staff	`password123`
`viewer`	Viewer	`password123`

These accounts are intended for testing authentication and role-based access restrictions during system development.

Security Note: The listed credentials are development test credentials and should be replaced with secure passwords before production deployment.

Summary

The system's role-based access control separates application permissions from PostgreSQL database privileges. Administrator and Staff accounts are granted full operational access within the web application, while Viewer accounts are limited to read-only access.

Authentication and authorization are handled through PHP sessions and server-side permission checks, while PostgreSQL group roles provide an additional layer of database access control.

This configuration establishes a foundation for secure user management, controlled database operations, and future expansion of role-specific system features.

Implementation Verification: The application-level permissions and database-level privileges must be tested against the actual source code and PostgreSQL configuration before being reported as fully implemented. The database accounts and group-role privileges listed above represent the intended configuration unless their creation and permissions have been independently verified.

7. Issues and Resolutions

The following issues were observed during development/setup. Update each status after confirming the fix in the running system.

ISSUE	RESOLUTION / ACTION	STATUS
Interviews page returned HTTP 500 because PHP queried i.result while the database column was results.	Changed the interview query to use results (or alias it for PHP display).	Reported working after the change.
Applications page reported that Pending application status was missing.	Seed script includes Pending, but its transaction did not complete. Load corrected seed data into the same database used by the app.	Not yet confirmed resolved.
Seed insert rejected position status On Hold due to chk_position_status.	Inspected constraint; allowed values are Open and Closed. Change sample value to Closed unless schema change is approved.	Full seed run not confirmed.
Seed insert failed because department_id 2 did not exist.	Inspect department IDs and resolve existing-record conflicts; ensure departments exist before positions.	Pending verification.
SQLSTATE 25P02: current transaction is aborted.	Roll back the failed transaction, fix the first failing statement, then rerun the corrected script.	Successful full seed run not confirmed.

Add additional errors, dates, fixes, and responsible team members here.

8. Cross-Group Verification

Complete this section using actual testing of another group's project and feedback received about the AMS. Do not invent results.

8.1 Other Group Project Tested

ITEM	DETAILS TO COMPLETE
Group / project	Group 01 / DropBoxed
Repository / system link	https://github.com/joshxaaaaaa/GRP1_Dropboxed
Date tested	November 6, 2026
Functions tested	Activity 4, Activity 5, and Activity 6
Results / issues	Activity 4 & 5 succeeded: Least-privilege roles were initialized, and environment variables were configured smoothly allowing the application to run on localhost:3000. Activity 6 succeeded: Backup archive verified and restored cleanly with constraints intact.
Feedback given	Logged as Success in the tracker. Great job organizing the database scripts and environment configuration for a smooth local deployment.

ITEM	DETAILS TO COMPLETE
Group / project	Group 02 / High School Event Management System
Repository / system link	https://github.com/JosephEmmanuelBabas/School_Event_Management_Web_Appli.git
Date tested	November 6, 2026
Functions tested	Activity 4, Activity 5, and Activity 6
Results / issues	Activity 4 succeeded. Activity 5 encountered issues: A schema permission restriction (`permission denied for schema public`) prevented Flyway from running. Activity 6 succeeded: Successfully handled schema ownership permissions during backup restoration.
Feedback given	Logged the schema permission issue in the tracker. Advised the group to grant proper schema creation privileges to the `event_app` role so the application can initialize its tables properly.

ITEM	DETAILS TO COMPLETE
Group / project	PAWPAW: Veterinary Appointment and Pet Management System
Repository / system link	https://github.com/rubiakylashane/DBA_PAWPAW.git
Date tested	November 6, 2026
Functions tested	Activity 4, Activity 5, and Activity 6
Results / issues	Success across all activities. Database roles and permissions were applied successfully (Act 4). Application configuration and environment startup were verified with no errors (Act 5). Backup archive was validated and restored cleanly (Act 6).
Feedback given	Logged as Success in the tracker. The database roles and environment configuration worked perfectly.

ITEM	DETAILS TO COMPLETE
Group / project	Group 04 / HealthBridge Clinic Management System
Repository / system link	https://github.com/kishaop/GRP03_PROJECTDB
Date tested	November 6, 2026
Functions tested	Activity 4, Activity 5, and Activity 6
Results / issues	Activity 4 succeeded: Created the database `grp03_projectdb`, executed `schema.sql` and `seed.sql`, and configured least-privilege role permissions for `hb_app`. Activity 5 encountered issues: The PHP app could not run in the browser (Apache returned a Not Found error). Activity 6 succeeded: Created backup and restored to a test database successfully.

Feedback given	Logged the browser access issue in the tracker. Advised the group to verify their folder structure and local Apache configuration so the application loads without returning a 404 error.

ITEM	DETAILS TO COMPLETE
Group / project	Group 5 / SHIORICafe
Repository / system link	https://github.com/LeeJnsn27/SHIORIcafe-new
Date tested	November 2 2026
Functions tested	Activity 4 (Database Setup), Activity 5 (Application Startup), and Activity 6 (Backup & Recovery)
Results / issues	Success across all activities. Established database schema and user privileges (Act 4). Built and ran the application server successfully without port conflicts (Act 5). Created and verified the backup archive `shioricafe_activity6.dump` and safely restored the inventory schema into a test database (Act 6).
Feedback given	Logged successful completion in the class tracker. Commended the group for well-configured environment variables and database scripts, which resulted in a smooth local deployment with zero errors.

ITEM	DETAILS TO COMPLETE
Group / project	Group 06 / Car Rental Management System (ByaheRentals)
Repository / system link	https://github.com/123airabels/Car_Rental_Management_System
Date tested	November 6, 2026
Functions tested	Activity 4, Activity 5, and Activity 6
Results / issues	Activity 4 succeeded: Roles and access controls mapped properly. Activity 5 failed: Application deployment failed due to conflicting port usage and database runtime credential configuration mismatches. Activity 6 succeeded: Validated backup archive and completed recovery simulation.
Feedback given	Logged port and credential issues in the tracker. Recommended checking the `.env` variables and ensuring their application's port isn't being blocked by another background service.

ITEM	DETAILS TO COMPLETE
Group / project	Group 08 / Catering Management System
Repository / system link	https://github.com/yami-sumi/catering-management-system.git
Date tested	November 6, 2026
Functions tested	Activity 4, Activity 5, and Activity 6
Results / issues	Success across all activities. Database roles and privileges set up cleanly (Act 4). Application configuration initialized successfully without errors (Act 5). Backup verified and successfully restored to an isolated test database (Act 6).
Feedback given	Logged as Success in the tracker. The system was easy to set up and everything worked flawlessly straight out of the box.

ITEM	DETAILS TO COMPLETE
Group / project	Group 09 / Batumbakal Gym Membership System
Repository / system link	https://github.com/louisandreidg/Batumbakal-Gym
Date tested	November 6, 2026
Functions tested	Activity 4, Activity 5, and Activity 6

Results / issues	Activity 4 succeeded: Roles and permissions applied. Activity 5 encountered issues: Application startup failed due to a Spring Boot configuration error causing the backend to crash. Activity 6 succeeded: Backup constraints verified and data recovery test simulation completed.
Feedback given	Logged the Spring Boot issue in the tracker. Advised the team to double-check their `application.properties` configuration and dependencies to resolve the startup crash.

ITEM	DETAILS TO COMPLETE
Group / project	Group 10 / UniSync
Repository / system link	https://github.com/CoffyJosh/unisync.git
Date tested	November 6, 2026
Functions tested	Activity 4, Activity 5, and Activity 6
Results / issues	Success across all activities. Implemented PostgreSQL roles and access privileges properly (Act 4). System setup and database connections worked perfectly, with all intended modules functioning (Act 5). Verified PostgreSQL backup and restored it seamlessly into a test database (Act 6).
Feedback given	Logged as Success in the tracker. Excellent documentation and a very smooth deployment process with no errors encountered.

8.2 Feedback Received About Group 7 AMS

ITEM	DETAILS TO COMPLETE
Group that tested AMS	Group 01 / DropBoxed
Date tested	November 6, 2026
Feedback received	Setup worked and 9/9 app checks passed, but noted that the project README instructs connecting as the `postgres` superuser.
Changes made	[To be completed by your group: e.g., Updated the README to instruct users to connect using the restricted `ams_app_user` instead.]

ITEM	DETAILS TO COMPLETE
Group that tested AMS	Group 02 / High School Event Management System
Date tested	November 5, 2026
Feedback received	Reported encountering an error in an SQL query file during setup, and recommended adding a PHP setup guide in the README.
Changes made	[To be completed by your group: e.g., Fixed the SQL syntax error and added standalone PHP setup instructions to the repository README.]

ITEM	DETAILS TO COMPLETE
Group that tested AMS	Group 03 / PAWPAW
Date tested	November 6, 2026
Feedback received	During Activity 5 health checks, they found SQL errors in the logs caused by a `result/results` column mismatch. During Activity 6, they also noted resolving CONNECT privilege and PATH issues to complete the backup/restore.
Changes made	[To be completed by your group: e.g., Updated the PHP interview query to use `results` to match the database column, and granted explicit CONNECT privileges to the backup role.]

ITEM	DETAILS TO COMPLETE
Group that tested AMS	Group 05 / SHIORICafe
Date tested	November 7, 2026
Feedback received	Provided highly positive feedback on Activity 3: "Clear documentation with XAMPP and standalone PHP setup steps. Database setup succeeds via psql... Database connection via PDO works reliably with .env settings. Full CRUD access... Viewer: Strictly read-only."
Changes made	No fixes required. We verified that our `01_schema.sql`, `02_seed.sql`, and `.env` setups are working smoothly for external testers.

ITEM	DETAILS TO COMPLETE
Group that tested AMS	Group 11 / ReserveIT Classroom Reservation System
Date tested	November 6, 2026
Feedback received	Reported encountering HTTP 503 database connection failures, which they resolved by manually configuring the `.env` settings to match their local database credentials (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`).

Changes made	[To be completed by your group: e.g., Updated the `.env.example` file to make credential placeholders more obvious for first-time setup.]

9. AI Usage Declaration

ChatGPT was used as an AI assistance tool during development and preparation of this document. Assistance included PHP implementation guidance and debugging, PostgreSQL/SQL setup and troubleshooting, interpreting database errors, explaining workflows, and drafting/structuring documentation.

AI output was treated as assistance, not automatically accepted as correct. Suggested code and SQL were checked against project files, schema details, database error messages, and observed application behavior. Errors or mismatches found during testing were corrected or marked for further verification.

9.1 AI Tool and Validation Record

AI TOOL	TASKS PERFORMED	VALIDATION
ChatGPT	PHP coding guidance, debugging, SQL/database troubleshooting, workflow explanations, and documentation drafting.	Reviewed suggestions against source files and PostgreSQL errors; ran the app and checked results. Unverified items are marked pending.

The group should review this declaration and add any other AI tools or uses made by any member before submission.